AtomicAdd Audit: GPU Kernel Atomics

Every atomic operation in the BFS oblique-split pipeline — why it exists, what it costs, and whether it can be eliminated. • October 2026

TOTAL ATOMIC SITES	AVOIDABLE	HOT PATH SHARE	NEGLIGIBLE
10	**3**	**~4 ms**	**5**
across 2 files, 7 kernels	warp-coalesce or restructure	histogram is the dominant user	class counts, bootstrap, init

1. Histogram bins (hot) 2. Histogram merge (hot) 3. Partition counters 4. Class counts
5. ComputeOffsets 6. SplitUpdate counters 7. SplitUpdate class counts 8. FinalMinMax
9. Bootstrap dedup 10. CountRows

1 Histogram bin accumulation

oblique_apply_projection.cu : 1434 HOT PATH

```
atomicAdd(&sh_hist[lbl * num_bins + bin], d_ex_dup_counts[example_idx]);
```

WHAT IT DOES

Each thread in `BuildHistogramRandomKernel_2D` maps its row's projected value to a bin via `cub::LowerBound`, then increments the shared-memory histogram for that bin. With 256 threads and 32 bins, ~8 threads contend per bin on average.

WHY ATOMIC IS USED

Multiple threads within the same block can land in the same bin. Without an atomic, concurrent writes to the same shared-memory location produce a race and lost updates. This is the most-executed atomic in the pipeline — called once per active row per projection, per depth.

MEASURED COST

DEPTH	ROWS	NODES	HIST (MS)	CONTENTION
D1	662,834	1	0.696	8 thr/bin
D2	662,834	2	0.734	8 thr/bin
D3	662,834	4	0.785	8 thr/bin
D4	662,834	8	0.853	8 thr/bin
D5	662,834	16	0.947	8 thr/bin
Total			**4.015**	

CAN IT BE AVOIDED? PARTIALLY

Warp-privatized histograms. Each of the 8 warps in a block maintains its own 32-bin histogram in shared memory (8 warps × 2 classes × 32 bins = 512 ints = 2 KB). Within a warp, threads that hit the same bin use `__reduce_add_sync` (SM 80+) to sum before a single write — eliminating intra-warp contention entirely. After all warps finish, a single pass merges the 8 per-warp histograms into the final shared histogram with no atomics needed (each thread owns one slot).

Expected improvement: reduces contention from 8-way to zero within each warp, and the merge is a simple sum. On SM 89 (Ada), shared-memory atomics already have ~5-cycle latency, so the gain is moderate — estimated 10–20% on the histogram kernel.

Ballot-based histogram. For each of the 32 bins, every warp executes `__ballot_sync(mask, my_bin == b)` + `__popc`. Eliminates atomics entirely but requires 32 ballot rounds per warp — trades atomic latency for instruction count. Only wins if bins are few (≤ 32) and contention is high.

2 Histogram merge to global

oblique_apply_projection.cu : 1447–1448 HOT PATH

```
if (c0) atomicAdd(&d_hist_class0[off], c0);
if (c1) atomicAdd(&d_hist_class1[off], c1);
```

WHAT IT DOES

After all threads in a block accumulate into the shared-memory histogram, threads 0–31 merge the block's partial histogram into the global histogram arrays. Multiple blocks may contribute to the same (node, projection) segment.

WHY ATOMIC IS USED

At depth 1 with 1 node, all ~2,589 blocks write to the same segment. Each block's thread 0–31 needs to add its partial bin count to the global total. Without an atomic, concurrent blocks overwrite each other.

CONTENTION

DEPTH	BLOCKS/SEGMENT	CONTENTION PER BIN
D1 (1 node)	2,589	2,589 blocks
D3 (4 nodes)	~647	~647 blocks
D5 (16 nodes)	~162	~162 blocks

CAN IT BE AVOIDED? PARTIALLY

Two-pass merge. Each block writes its per-block partial histogram to a staging buffer (no atomics — each block owns its slot). A second kernel reduces all partial histograms per segment with a tree reduction. Eliminates global atomics entirely but adds a second kernel launch + staging memory.

Verdict: The two-pass approach adds complexity for a small gain. Global L2 atomics on SM 89 handle this well. The `if (c0)` / `if (c1)` guards already skip zero-count bins. **Not worth optimizing.**

3 Partition position counters

oblique_apply_projection.cu : 290, 293 PARTITION

```
int pos_idx = atomicAdd(&d_atomic_counters[node_id], 1);      // right-child slot
int neg_idx = atomicAdd(&d_atomic_counters[num_nodes + node_id], 1); // left-child slot
```

WHAT IT DOES

In `SplitExamplesInPlaceKernel`, each thread decides whether its row goes left or right based on the split threshold, then atomically claims a write position in the child's segment of `d_selected_examples`.

WHY ATOMIC IS USED

Classic scatter pattern: all threads in a node race to write to the same two counters. The atomic ensures each row gets a unique position.

CAN IT BE AVOIDED? YES

Already avoided in the StablePartition path. `StablePartitionCountKernel` + `StablePartitionScatterKernel` use a two-pass prefix-scan approach with no position atomics. The unstable partition is the current default because output order doesn't matter. Measured difference is negligible — only `num_nodes` distinct counters spread across L2 cache lines.

4 Partition class counts

oblique_apply_projection.cu : 299, 422 PARTITION

```
atomicAdd(&d_class_counts[node_id * 2 * num_classes + side * num_classes + cls],
          d_ex_dup_counts[example_idx]);
```

WHAT IT DOES

Each thread adds its row's class weight to the appropriate child's class-count bucket. Binary classification: only `num_nodes × 4` distinct counters.

CAN IT BE AVOIDED? PARTIALLY

Warp-coalesced atomics or **shared-memory pre-accumulation** could reduce calls by ~16×. But measured cost is negligible — L2 atomic coalescing handles the few counters efficiently. **Not a bottleneck.**

---

5 ComputeOffsets position counters

node_id_drsw.cuh : 105 `DRSW_DEPTH`

```
base_offset = atomicAdd(&d_atomic_counters[nid], group_size);
```

WHAT IT DOES

In `NodeIdDRSW_ComputeOffsetsKernel`, each warp groups threads by node via `__match_any_sync`, sums the group, and the leader atomically claims positions for the entire group.

ALREADY OPTIMIZED

Warp-coalesced pattern reduces atomics from ~662K (one/thread) to ~20K–40K (one/warp-group). At depth 1 with 1 node: 1 atomic per 32 active threads.

MEASURED COST

Total `drsw_depth` = 0.34 ms across 5 depths (0.07 ms/depth). **Leave as is.**

6 SplitUpdate child counters

node_id_drsw.cuh : 261 `PARTITION`

```
atomicAdd(&d_child_counters[child_id], 1);
```

WHAT IT DOES

Each thread in `NodeIdDRSW_SplitUpdateKernel` increments the child's row counter after writing `node_ids[row] = child_id`.

CAN IT BE AVOIDED? `YES`

Warp-coalesced atomics (same `__match_any_sync` pattern as #5) would reduce calls by ~16×. Or run a separate `NodeIdDRSW_CountRows` kernel instead. Saving: ~0.02–0.05 ms/depth.

7 SplitUpdate class counts

node_id_drsw.cuh : 269 `PARTITION`

```
atomicAdd(&d_child_class_counts[child_id * num_classes + cls], weight);
```

Same analysis as #4. Only `2 × num_children × 2` counters. **Not a bottleneck.**

---

8 FinalMinMax reduction

oblique_apply_projection.cu : 1118–1119, 1183–1184 `FINMM`

```
atomicMinFloat(&outMin[seg], block_mm.minv);
atomicMaxFloat(&outMax[seg], block_mm.maxv);
```

WHAT IT DOES

Merges per-block min/max into per-segment (node × projection) min/max via CAS-based float atomics.

CONTENTION

At D1: only ~11 meta-blocks contend per segment. CAS loop almost never retries. `finMM` = 0.01–0.03 ms/depth. **Negligible.**

9 Bootstrap dedup counts

oblique_apply_projection.cu : 218 `ONE-TIME`

```
atomicAdd(&d_ex_dup_counts[d_bootstrap[tid]], 1);
```

Counts how many times each row appears in the bootstrap sample. Run once per tree. Part of `dedup` = 0.49 ms total. Low contention (most rows sampled 0–2 times). **Not worth optimizing.**

10 CountRows histogram

node_id_drsw.cuh : 437 `INIT ONLY`

```
if (nid >= 0 && nid < num_nodes) atomicAdd(&counts[nid], 1);
```

Simple row-count histogram over `d_node_ids`. Runs at most once per tree during initialization. **Not worth touching.**

Priority ranking

Ranked by wall-time impact. Measured on 1,048,576 × 1,024 synthetic dataset, tree depth 6, 1 tree, SM 89 (RTX 4090).

#	SITE	TIME/DEPTH	TOTAL (5 DEPTHS)	AVOIDABLE?	ESTIMATED SAVING
1	Histogram bin accumulation	0.70–0.95 ms	4.02 ms	PARTIAL	0.4–0.8 ms (warp-private hist)
2	Histogram global merge	incl. above	incl. above	PARTIAL	minimal (L2 atomics fast)

6	SplitUpdate child counters	incl. in 0.37 ms	1.94 ms*	YES	0.1–0.25 ms (warp-coalesce)
3	Partition position counters	incl. in 0.69 ms	3.46 ms*	YES	0 (stable partition exists)
5	ComputeOffsets counters	0.07 ms	0.34 ms	YES	~0 (already warp-coalesced)
4,7	Class counts (both paths)	< 0.01 ms	negligible	PARTIAL	~0
8	FinalMinMax	0.01 ms	0.05 ms	YES	~0
9	Bootstrap dedup	one-time	0.49 ms	NO	~0
10	CountRows	init only	negligible	NO	~0

* Total includes non-atomic work (projection reads, scatter writes, label lookups). The atomic itself is a fraction of this total.

Recommendation

The only atomic worth optimizing is #1 — histogram bin accumulation.

It accounts for ~4 ms out of the ~28 ms depth-loop, and it's the only site where contention is both high (8 threads/bin avg) and on the hot path (every row × every projection × every depth). All other atomics are either already warp-coalesced (#5), low-contention (#3, #4, #6, #7, #8), or one-time costs (#9, #10).

The highest-ROI improvement is **warp-privatized histograms**: give each of the 8 warps its own 32-bin shared-memory histogram. Threads within a warp accumulate via `__reduce_add_sync` (single-cycle on SM 80+). After the warp loop, a merge pass sums the 8 partial histograms — 256 threads merging 512 values, zero contention. Shared-memory cost: 8 × 2 × 32 × 4 = 2,048 bytes on top of the existing 256 bytes for splits. Well within the 48 KB shared-memory budget.